

基于强化学习与蒙特卡洛树搜索的资源约束量子布尔函数综合方法

中文论文稿 v13: frontier policy 与 truth-table bridge 版

2026 年 7 月 8 日

摘要

量子布尔函数 oracle 综合是把经典谓词接入量子算法的核心编译步骤。传统综合路线通常在布尔表示、可逆逻辑分解或 CNOT 导向覆盖空间中优化单一资源，但容错量子计算中的实际逻辑开销同时受 T-count、CNOT、深度、总门数和辅助比特影响。本文将 oracle 综合表述为代数正规形 (algebraic normal form, ANF) 与固定极性 Reed-Muller (fixed-polarity Reed-Muller, FPRM) 项集合上的资源约束搜索问题，提出结合神经动作先验、蒙特卡洛树搜索、布尔环线性因子筛选、结构级 depth policy、depth-frontier policy 和保守 guard 的 Resource-NMCTS 框架。与 SSHR 不同，本文方法不枚举 parallelotope cover；SSHR 仅作为 small-scale CNOT-oriented baseline。实验表明，在 $n \leq 6$ 的 177 个传统函数上，Pareto-Resource-NMCTS 相对 ESOP cube beam 获得 174/0/3 的 score 胜/负/平，平均 score 降低 36.09%；相对 weighted ESOP-MILP 获得 167/3/7，平均 score 降低 29.84%。在 $n = 18$ 的随机 ANF 函数上，完整 Resource-NMCTS 相对 adaptive depth-2 Boolean-ring screen 进一步降低 23.85% 平均 score；在 $n = 20$ 边界函数上，screen-gated Resource-NMCTS 与完整 Resource-NMCTS 资源持平并把平均运行时间从 77.10 s 降至 20.71 s。在 $n = 20, 22, 24, 28, 32, 36, 40$ 的项集级 scale 测试中，depth policy 相对 single screen 稳定降低 5.55%–6.56% 平均 score，并相对 all-depth adaptive 节省约三分之一时间。进一步的 depth-frontier 实验显示，depth-4 Boolean-ring screen 相对 fixed depth-2 在 $n = 20, 28, 40$ 上平均 score 再降低 3.10%；新训练的 depth-frontier policy 在同一切片相对 fixed depth-2 获得 35/0/37、平均 score 降低 2.19%，并相对 all-depth depth ≤ 4 评估节省 58.69% 时间。当前引用的项集级结果文件中，3720/3720 个方法行均通过 factor-plan ANF 展开验证和 emitted X/CNOT/MCT circuit 的 GF(2) 多项式符号模拟验证。为弥合符号验证和完整枚举之间的边界，本文又在 $n = 21, 22$ 的 12 个生成式 ANF 函数上构造完整 truth-table，120/120 个方法行同时通过 truth-table oracle 验证、plan ANF 验证和 emitted-circuit 符号验证。本文的阶段性结论是：该路线已经形成独立于 SSHR 的逻辑层低 T-count/低加权资源综合主线，并已补上高预算质量前沿学习化与 $n > 20$ 完整验证桥接；最终投稿仍需补充更强的高维 RL/MCTS 独立质量收益和外部 reversible-toolchain 对比。

关键词：量子 oracle 综合；布尔函数综合；强化学习；蒙特卡洛树搜索；ANF；FPRM；布尔环；资源约束综合

1 引言

给定布尔函数 $f : \{0, 1\}^n \rightarrow \{0, 1\}$, 量子 oracle 通常实现为

$$|x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle. \quad (1)$$

这类 oracle 出现在 Grover 搜索、密码分析、约束满足、组合优化和若干量子算法子程序中。若 f 的综合线路被直接展开为单项式逐项写入输出位, 语义验证简单, 但会丢失可共享的代数结构; 若只优化 CNOT 或只压缩 AND 节点, 又可能增加 T 门、辅助比特或调度深度。因此, 本文把量子布尔函数综合视为一个多资源组合搜索问题, 而不是单一门数最小化问题。

本文采用 ANF 作为基础表示:

$$f(x) = \bigoplus_{m \in \mathcal{M}} \prod_{i \in m} x_i. \quad (2)$$

ANF 的优势是语义透明, 任一项都能映射为受控乘法结构; 缺点是原始项集合不显式给出公共因子、线性奇偶量、极性重写和可递归分解的子问题。已有 XAG、ROS、BDD 与 SSHR 等工作从不同中间表示改进 oracle 或可逆逻辑综合 [2, 3, 4, 6]。这些方法说明表示选择会显著影响资源, 但并未直接回答一个问题: 能否在 ANF/FPRM 项集合上定义可搜索的代数动作, 并用 AI 策略选择更有资源收益的综合路径?

本文的一句话论证是: 在逻辑层量子布尔函数 oracle 综合中, ANF/FPRM 项集合上的资源感知神经搜索, 配合布尔环线性因子筛选和保守结构门控, 可以在传统函数与高维项集合上降低 T-count 和加权逻辑资源; 当前边界是该方法尚未覆盖硬件 mapping、真实噪声模型和完整的大规模 truth-table 枚举验证。这个表述也决定了本文和 SSHR 的关系: SSHR 是 CNOT-oriented small-scale baseline, 而不是本文方法的出发点。

2 相关工作与定位

2.1 布尔表示驱动的 oracle 综合

Xor-And-Inverter Graphs (XAG) 将 XOR 与 AND 结构分离, 使 AND 节点数量与非 Clifford 成本直接相关。Meuli 等人将 XAG 用于 quantum compilation [2], 并从 multiplicative complexity 角度讨论低 T-count oracle 的结构来源 [1]。ROS 进一步把 oracle synthesis 写成 resource-constrained LUT mapping [3]。本文与这些工作共同强调布尔结构对逻辑资源的决定作用, 但本文不固定使用 XAG 或 LUT, 而是在 ANF/FPRM 项集合上搜索可计算、可反算的因子结构。

BDD-based reversible synthesis 可以处理较大的布尔函数 [4]。Back-end-aware oracle synthesis 则把 XAG 优化与后端质量指标连接起来 [5]。这些工作覆盖了本文尚未完成的后端维度。本文当前只报告逻辑层 T-count、CNOT、depth、gate count 和 peak ancilla, 不包含 routing、magic-state factory、连通性约束或噪声下保真度。

SSHR 使用 parallelotope 的空间结构做 small-scale Boolean function 的 CNOT-oriented synthesis [6]。本文保留 SSHR-H/SSHR-I 作为比较对象, 但不继承其 candidate 空间。更准确地说, 本文是一条面向低 T-count 和加权逻辑资源的项集合搜索路线, SSHR 在这里只用于说明 CNOT 指标上的 trade-off。

表 1: 本文的核心术语与边界。

术语	本文含义
Resource-NMCTS	在 ANF/FPRM 项集合上进行资源加权搜索的主框架, 包括神经先验、MCTS、guard 和 portfolio 选择。
Boolean-ring screen	在 square-free Boolean ring 中搜索可复用线性因子的结构筛选分支, 允许线性因子与 quotient 共享变量。
Depth policy	预测 single、depth-1 或 depth-2 screen 的结构级神经策略。
Depth-frontier	使用 depth-3/4 Boolean-ring screen 追求更低 score 的高预算质量模式。
Depth-frontier policy	在 depth-2/3/4 之间直接选择的结构级策略, 用于接近 depth-4 质量并减少 all-depth 评估时间。
Truth-table bridge	在 $n = 21, 22$ 上构造完整 truth-table, 把项集级符号验证与函数级完整验证连接起来。
SSHR	CNOT-oriented small-scale baseline; 本文方法不从 SSHR 搜索空间出发。

2.2 学习式搜索与线路优化

学习式搜索已经用于多个离散综合任务。ShortCircuit 采用 AlphaZero 风格搜索设计经典布尔电路 [7]; Gumbel AlphaZero 被用于 Clifford+T unitary synthesis[8]; ZX-calculus 优化中出现了强化学习和图神经网络驱动的 rewrite-rule 选择 [9]; MonteQ 使用 MCTS 处理量子线路综合的动作排序 [10]。这些工作说明, 学习策略适合处理组合动作空间。本文的差异在于状态、动作和奖励都围绕 Boolean oracle 的 ANF/FPRM 因子分解定义, AI 模块主要承担动作排序、结构深度选择和安全门控, 而不是直接生成任意量子门序列。

3 问题定义与资源模型

本文输入为布尔函数的 truth-table 或 ANF 表示, 输出为实现式 (1) 的逻辑层可逆线路。候选线路由 X、CNOT 和多控 Toffoli (MCT) 抽象门组成, 并在资源统计阶段转化为 T-count、CNOT、深度、总 gate count 和 peak ancilla。用于搜索排序的统一分数定义为

$$\text{score} = T + 0.04 \text{ CNOT} + 0.015 \text{ depth} + 0.01 \text{ gates} + 2 \text{ ancilla}. \quad (3)$$

该分数用于比较候选逻辑线路, 不代表某个具体设备的物理时间或错误率。为避免单一分数掩盖 trade-off, 实验同时报告 T-count、CNOT、depth、gate count 和 peak ancilla。

在 $n \leq 20$ 的主函数级实验中, 候选线路通过完整 truth-table 语义验证; 本文另设 $n = 21, 22$ bridge 切片, 专门验证高维项集方法在完整 truth-table 枚举下是否仍正确。在更大的 $n > 22$ 项集级 scale 实验中, 完整 truth-table 构造本身不可接受, 因此采用两层符号验证: 第一层将 factor plan 展开回 ANF 项集合, 检查目标项集合等价; 第二层对 emitted X/CNOT/MCT circuit 做 GF(2) 多项式符号模拟, 检查输入线复原、输出线等价和辅助线清零。该验证不等同于逐点枚举 2^n 个输入, 但强于只统计 plan 资源。

4 方法

4.1 状态、动作与搜索流程

Resource-NMCTS 的状态是当前尚未综合的项集合 $\mathcal{M}$ 。一次动作选择一个可计算、可反算的局部代数结构, 将 $\mathcal{M}$ 分解为 quotient 子问题和 rest 子问题, 再递归生成 compute/action/uncompute 形式的线路。候选动作包括公共单项式因子、FPRM 极性重写后的公共项、线性奇偶因子、布尔环线性因子, 以及由神经模型扩展的 root actions。

神经动作先验不直接输出量子门序列, 而是对候选因子动作排序。特征包括候选覆盖率、项数变化、项平均次数、变量覆盖密度、即时资源收益和子问题规模等结构统计。MCTS 在有限预算内探索候选组合, rollout 使用确定性资源估计与已有 baseline 候选。最终选择由 baseline-preserving guard 兜底: 若学习候选不优于确定性 baseline, 则返回 baseline。这个设计牺牲部分探索自由度, 但降低了学习模型在分布外项集合上造成资源退化的风险。

4.2 布尔环线性因子筛选

早期 linear-pair 分支主要搜索

$$(x_i \oplus x_j \oplus \dots) \cdot h(x) \quad (4)$$

形式的结构, 并要求线性因子变量与 quotient $h(x)$ 的变量不相交。该约束便于实现, 却排除了 square-free Boolean ring 中合法的重叠变量情形。本文的 Boolean-ring screen 允许二者共享变量, 并在展开时使用 $x_i^2 = x_i$ 与 GF(2) 偶数项消去。例如

$$(x_i \oplus x_j)x_im = x_im \oplus x_ix_jm. \quad (5)$$

线路层面仍可先用 CNOT 计算线性奇偶量, 再用该辅助量控制 quotient 子线路。因此, 布尔环线性因子不是简单扩大 beam 宽度, 而是把原先被实现约束排除的代数候选重新纳入可综合搜索空间。

4.3 结构级 AI 与保守门控

高维实验显示, single screen、depth-1 screen 和 depth-2 screen 的收益依赖项集合结构。本文因此训练 depth policy, 输入项数、次数分布、变量覆盖密度、二元共现密度和 root Boolean-ring action 统计, 输出应该使用的 screen 深度。监督标签来自实际运行三种 screen 后按式 (3) 选择的最佳 depth。

固定 depth-2 screen 是强质量 baseline, 因此本文又训练保守 depth-2 skip guard。Static-direct 模式只使用项集合静态特征, 若预测 depth-1 足以与 fixed depth-2 持平, 则直接运行较浅 screen; 否则运行 depth-2。阈值按 zero-false-skip 约束选择, 使门控优先避免质量损失。类似地, $n = 20$ 边界切片中完整 Resource tail 与 adaptive screen 资源持平但运行更慢, 本文训练 screen-gate 判断是否可以跳过 Resource tail。当前 screen-gate 只作为运行时控制证据, 不作为高维质量提升的主证据。

表 2: $n \leq 6$ 传统函数切片上的平均逻辑资源。

方法	平均 T	平均 CNOT	平均 depth	平均 ancilla	平均 score
Direct ANF	184.1	134.2	134.6	1.33	194.3
AND-direct ANF	101.0	166.5	166.9	2.18	115.2
Resource-NMCTS	43.9	89.9	94.5	1.92	53.2
Pareto-Resource-NMCTS	40.4	83.0	88.0	2.03	49.6
ESOP-MILP	83.6	133.5	159.6	2.32	96.7
SSHR-H	81.0	67.6	88.7	1.36	88.2

在高预算设置下, fixed depth-2 不再是质量上限。本文进一步训练 depth-frontier policy, 使模型在 depth-2、depth-3 和 depth-4 Boolean-ring screen 之间选择。训练标签来自三种深度的实际资源最优项, 目标不是超过 oracle frontier, 而是在接近 depth-4 质量的同时避免 all-depth depth ≤ 4 的完整枚举开销。

5 实验设计

实验分为六组。第一组使用 $n \leq 6$ 的 177 个传统函数, 与 direct ANF、AND-direct ANF、ESOP cube beam、weighted ESOP-MILP、SSHR-H、ABC 和 BDD baseline 比较。第二组使用 $n = 18$ 随机 ANF 函数, 观察 adaptive screen 与完整 Resource-NMCTS 的差距。第三组使用 $n = 20$ 边界函数, 检验在 FPRM root beam 和 fast linear-pair 超时区域中, depth-2 screen 是否仍能给出可运行改进, 并用 $n = 19, 20$ held-out 切片验证 screen-gate 是否会错误跳过 Resource tail。第四组使用 $n = 14, 16, 18$ 生成式项集训练 depth policy 与 skip guard, 并在 held-out $n = 20$ 项集上测试结构选择能力。第五组在 ANF 项集合上评估 $n = 20, 22, 24, 28, 32, 36, 40$ 的 scale 行为, 并对每个 plan 和 emitted circuit 做符号等价验证; 同组还训练 depth-frontier policy, 把 depth-3/4 的高预算质量收益学习化。第六组在 $n = 21, 22$ 上构造完整 truth-table bridge, 对生成的 X/CNOT/MCT oracle circuit 做逐点语义验证。

所有 paired comparison 只纳入函数名或项集名匹配、语义正确且未 timeout 的结果。Timeout 行保留为运行边界证据, 但不参与正确结果均值比较。本文所有实验均为逻辑层资源估计, 不包含 hardware mapping 和 noise-aware scheduling。

6 结果

6.1 小规模传统函数

表 2 给出 $n \leq 6$ 传统函数上的平均逻辑资源。相对 direct ANF, Pareto-Resource-NMCTS 平均 T-count 降低 72.25%, 平均 score 降低 67.80%。相对 ESOP cube beam, Pareto-Resource-NMCTS 获得 174/0/3 的 score 胜/负/平, 平均 score 降低 36.09%。相对 weighted ESOP-MILP, 获得 167/3/7, 平均 score 降低 29.84%。

该结果支持本文的低 T-count 与加权资源优势, 但也给出重要边界: SSHR-H 的平均 CNOT 最低, 因此本文不能主张 CNOT-only 指标全面支配 SSHR。更准确的结论是, 本文方法在当前资

表 3: 高维函数级边界结果。

切片	方法	平均 T	平均 CNOT	平均 score	平均时间 (s)
$n = 18$	Single screen	10389.33	16268.42	11349.06	0.82
$n = 18$	Adaptive depth-2 screen	9767.00	15301.58	10670.94	4.39
$n = 18$	Resource-NMCTS	6639.33	11380.92	7315.62	226.18
$n = 20$	AND-direct ANF	21516.67	33635.67	23491.15	10.41
$n = 20$	Single screen	20786.00	32492.50	22695.15	10.70
$n = 20$	Adaptive depth-2 screen	19535.33	30542.50	21331.24	20.67
$n = 20$	Resource-NMCTS	19535.33	30542.50	21331.24	77.10
$n = 20$	Screen-gated Resource-NMCTS	19535.33	30542.50	21331.24	20.71

源权重下用一定 CNOT 与辅助比特 trade-off 换取更低 T-count 和更低 score。

6.2 高维函数级边界

在 $n = 18$ 的 12 个随机 ANF 函数上, adaptive depth-2 Boolean-ring screen 相对 single screen 获得 11/0/1 的 score 胜/负/平, 平均 score 从 11349.06 降至 10670.94; 完整 Resource-NMCTS 平均 score 为 7315.62, 相对 adaptive screen 进一步降低 23.85%。这说明 $n = 18$ 上快速 screen 是有效候选生成器, 但不能替代完整资源搜索。

在 $n = 20$ 边界函数上, FPRM root beam 和 fast linear-pair 均触发 300 s task timeout。Adaptive depth-2 screen 相对 single screen 获得 5/0/1, 平均 score 降低 7.47%。集成该分支后, Resource-NMCTS 相对 AND-direct ANF 获得 5/0/1, 平均 score 降低 11.80%。由于此切片中完整 Resource tail 与 adaptive screen 资源持平, screen-gated Resource-NMCTS 可以把平均时间从 77.10 s 降到 20.71 s。

6.3 结构级 AI 结果

Depth policy 在 $n = 14, 16, 18$ 项集上训练, 并在 held-out $n = 20$ 项集上评估。Policy 相对 single screen 获得 42/0/6, 平均 score 降低 6.57%; 相对 depth-1 screen 获得 34/0/14, 平均 score 降低 2.57%。相对 all-depth adaptive, policy 平均 score 只高 0.28%, 但平均运行时间降低 34.71%。这说明结构级 AI 已能学习 screen 深度选择, 但尚未在 score 上超过 fixed depth-2。

保守 depth-2 skip guard 去掉了这一质量缺口。Static-direct 模式在 held-out $n = 20$ test 中没有 false skip, 96 个样本全部与 fixed depth-2 screen score 持平, 并相对 fixed depth-2 与 all-depth adaptive 分别节省 0.54% 和 24.74% 平均时间。Screen-gate 的独立 held-out 验证也支持保守门控: 在 $n = 19, 20$ 共 16 个函数上, screen-gated Resource-NMCTS 与完整 Resource-NMCTS 全部 score 持平, 平均节省 36.83% 时间; 若直接用 adaptive screen 替代 Resource tail, $n = 19$ 子集会出现 4/8 个 score loss。

6.4 项集级 scale 与 depth-frontier

表 4 汇总 $n = 20, 22, 24, 28, 32, 36, 40$ 的项集级结果。在 $n = 20, 22, 24, 28$ 上, depth policy 相对 single screen 获得 169/0/23, 平均 score 降低 6.56%; 相对 all-depth adaptive 只有 0/6/186, 平均 score 增加 0.08%, 但平均时间降低 31.01%。在 $n = 32, 36, 40$ 上, depth policy 相对 single

screen 获得 110/0/34, 平均 score 降低 5.55%; 相对 all-depth adaptive 在 144 个项集上全部 score 持平, 并节省 33.14% 平均时间。

表 4: 项集级 screen-scale 结果。所有方法行均通过 plan 和 emitted-circuit ANF 符号验证。

范围	方法	baseline	score 胜/负/平	平均 Δ score	平均 Δ time
20-28	adaptive all-depth	single screen	169/0/23	-6.63%	+3077.71%
20-28	depth policy	single screen	169/0/23	-6.56%	+2328.66%
20-28	depth policy	all-depth adaptive	0/6/186	+0.08%	-31.01%
32-40	adaptive all-depth	single screen	110/0/34	-5.55%	+2680.60%
32-40	depth policy	single screen	110/0/34	-5.55%	+2061.11%
32-40	depth policy	all-depth adaptive	0/0/144	+0.00%	-33.14%

前述策略偏向快速或中等预算。为判断 fixed depth-2 是否已经达到质量上限, 本文又在 $n = 20, 28, 40$ 的 72 个项集上运行 depth-3/4 Boolean-ring screen。表 5 显示, depth-3 相对 fixed depth-2 获得 49/0/23, 平均 score 降低 1.93%; depth-4 相对 fixed depth-2 同样获得 49/0/23, 平均 score 降低 3.10%。这说明更深布尔环筛选确实能形成新的质量前沿, 但代价是运行时间分别增加 193.71% 和 682.97%。因此, depth-3/4 应被写成高预算质量模式, 而不是默认快速策略。

表 5: $n = 20, 28, 40$ depth-frontier screen-scale 比较。

方法	baseline	score 胜/负/平	平均 Δ score	平均 Δ time
screen depth-3	fixed depth-2	49/0/23	-1.93%	+193.71%
screen depth-4	fixed depth-2	49/0/23	-3.10%	+682.97%
screen depth-4	screen depth-3	44/0/28	-1.21%	+127.55%
all-depth adaptive depth ≤ 4	fixed depth-2	49/0/23	-3.10%	+1129.85%

为把高预算质量前沿转化为 AI 策略, 本文训练 depth-frontier policy 在 depth-2/3/4 之间选择。表 6 显示, 在独立 held-out $n = 28, 40$ 的 32 个项集上, frontier policy 相对 depth-2 获得 13/0/19, 平均 score 降低 1.95%; 相对 depth-4 平均 score 只高 0.80%, 但节省 25.99% 时间; 相对 oracle depth-2/3/4 frontier 的 all-depth 评估, 平均 score gap 为 0.80%, 平均时间降低 58.76%。把该 policy 接入 $n = 20, 28, 40$ scale harness 后, 它相对 fixed depth-2 获得 35/0/37, 平均 score 降低 2.19%; 相对 all-depth adaptive depth ≤ 4 平均 score 高 0.97%, 但节省 58.69% 时间。因此, depth-frontier 已不再只是离线 oracle 质量前沿, 而是形成了一个可学习的质量-时间折中策略。

表 6: Depth-frontier policy 的 held-out 与 scale harness 结果。

设置	比较	score 胜/负/平	平均 Δ score	平均 Δ time
held-out $n = 28, 40$	policy vs depth-2	13/0/19	-1.95%	+455.93%
held-out $n = 28, 40$	policy vs depth-4	0/9/23	+0.80%	-25.99%
held-out $n = 28, 40$	policy vs oracle all-depth	0/9/23	+0.80%	-58.76%
scale $n = 20, 28, 40$	policy vs depth-2	35/0/37	-2.19%	+503.20%
scale $n = 20, 28, 40$	policy vs depth-4	0/16/56	+0.97%	-22.17%
scale $n = 20, 28, 40$	policy vs all-depth	0/16/56	+0.97%	-58.69%

主 scale、extended scale、depth-frontier 和 depth-frontier-policy rerun 四组项集级结果合计包含 3720 个方法行。所有方法行均通过两层验证: factor plan 的 ANF 展开验证为 3720/3720,

emitted X/CNOT/MCT circuit 的 GF(2) 多项式符号模拟验证为 3720/3720, mismatch 总数为 0。这使项集级结果不只是资源估计, 但仍应明确它不是完整 truth-table simulation。

6.5 完整 truth-table bridge

为进一步缩小 $n > 20$ 项集级符号验证和完整函数级验证之间的距离, 本文在 $n = 21, 22$ 上构造了 12 个生成式 ANF 函数, 并显式构造完整 truth-table。该 bridge 小于 $n = 20-40$ scale 测试, 因为 truth-table 构造是主要成本; 但每个方法行都生成 X/CNOT/MCT oracle circuit, 并用 bit-parallel oracle verifier 做逐点语义验证。表 7 显示, 120/120 个方法行通过完整 truth-table 验证、plan ANF 展开验证和 emitted-circuit GF(2) 符号验证, mismatch 为 0。该实验还复现了 depth-frontier 的质量信号: 在 $n = 21, 22$ 上, screen depth-4 相对 fixed depth-2 获得 10/0/2, 平均 score 降低 3.81%, 但 plan 时间增加 734.91%; depth-frontier policy 相对 fixed depth-2 获得 8/0/4, 平均 score 降低 3.50%, 相对 all-depth adaptive 节省 50.87% plan time。

表 7: $n = 21, 22$ 完整 truth-table bridge。

验证项或比较	结果	说明
完整 truth-table oracle 验证	120/120	所有 emitted circuit 逐点验证通过
Plan ANF 展开验证	120/120	0 plan mismatch
Emitted-circuit ANF 符号验证	120/120	0 circuit mismatch, max wire terms 215
平均 truth-table 构造时间	30.34 s/function	$n = 21, 22$ 合计 12 个函数
screen depth-4 vs fixed depth-2	10/0/2	平均 score -3.81%, plan time +734.91%
depth-frontier policy vs fixed depth-2	8/0/4	平均 score -3.50%, plan time +634.71%
depth-frontier policy vs all-depth	0/2/10	平均 score +0.32%, plan time -50.87%
adaptive all-depth vs single screen	11/0/1	平均 score -10.19%

7 证据地图与投稿缺口

表 8 给出当前主张和证据的对应关系。与 v12 相比, v13 的主要变化是: depth-frontier 的质量收益已经被学习策略部分吸收, $n = 21, 22$ 也已有完整 truth-table bridge。

表 8: 当前可支撑主张与缺口。

主张	证据	投稿状态
独立于 SSHR 的低 T-count/低 score 主线	$n \leq 6$ 传统函数相对 ESOP、weighted ESOP-MILP 和 direct baselines 的稳定优势; SSHR-H 保留为 CNOT trade-off 参照。	可写入摘要, 但需说明 CNOT-only 不支配 SSHR。
布尔环 screen 是高维有效结构筛选	$n = 18$ 、 $n = 20$ 函数级边界以及 $n = 20$ 到 $n = 40$ 项集级 scale 均显示相对 single screen 的 score 改善。	可作为核心方法贡献。
结构级 AI 能减少自适应搜索成本	Depth policy 与 skip guard 在 held-out 项集上接近或持平 all-depth adaptive, 并节省约三分之一时间。	可作为 AI 贡献, 但不宜夸大为强质量提升。
Depth-frontier 提供新的质量提升目标	Depth-4 相对 fixed depth-2 平均 score 再降低 3.10%, 但时间增加 682.97%。	可作为高预算质量前沿。
Depth-frontier policy 学习质量-时间折中	相对 fixed depth-2 为 35/0/37、平均 score -2.19%; 相对 all-depth depth ≤ 4 平均 score +0.97%, 时间 -58.69%。	可作为新增结构级 AI 质量证据。
高维 emitted circuit 语义可信	3720/3720 项集级方法行通过 plan 展开和 emitted circuit GF(2) 符号模拟验证。	可支撑项集级结果; truth bridge 进一步补强但样本仍小。
$n > 20$ 完整验证边界后移	$n = 21, 22$ 的 120/120 方法行通过完整 truth-table oracle 验证、plan 验证和 emitted-circuit 符号验证。	可作为完整验证桥接证据。
完整 RL/MCTS 在高维上形成强独立收益	当前高维 learned prior 独立质量收益仍弱, 部分切片主要由 screen/guard 驱动。	仍是投稿前主要缺口。

下一步最直接的提升目标已经从“证明 depth-frontier 存在”转为“把高预算质量进一步压缩成更便宜的策略”。具体来说, 应改进 depth-frontier policy 对 depth-4 oracle 的覆盖率, 减少当前相对 depth-4 的 0.97% 平均 score gap; 同时引入更强外部工具链对比, 尤其是 ROS、mockturtle/ABC 风格映射和 back-end-aware oracle synthesis, 使论文不只和内部 baseline 比较。

8 讨论

本文最稳妥的贡献表述有五点。第一, 方法主线独立于 SSHR。本文不是把 SSHR 改成 AI 版本, 而是在 ANF/FPRM 项集合上做资源加权搜索; SSHR 的作用是 CNOT-oriented baseline。第二, 布尔环线性因子提供了明确的结构扩展, 允许线性因子与 quotient 共享变量, 并在 $n = 18, 20$ 和项集级 scale 上稳定改善 single screen。第三, 结构级 AI 已有正向证据: depth policy 学会了 screen 深度选择, 保守 guard 消除了相对 fixed depth-2 的 score 缺口, 并减少 all-depth adaptive overhead。第四, depth-frontier 证明 fixed depth-2 不是质量上限, 而 depth-frontier policy 已把这一质量收益部分学习化。第五, truth-table bridge 把完整语义验证推进到 $n = 21, 22$, 使大规模项集级符号验证和函数级完整验证之间的断层变窄。

当前结果还不能支撑更强结论。其一, $n = 18$ 上完整 Resource-NMCTS 仍能超过 adaptive screen, 说明中等高维仍需要更深搜索; 其二, $n = 20$ 切片中完整 Resource-NMCTS 与 adaptive

screen 资源持平, 说明该切片收益主要来自结构筛选而非 MCTS tail; 其三, 高维 learned prior 的独立质量收益仍弱, 当前 AI 主体更多体现为结构选择和安全门控; 其四, $n = 24$ 到 $n = 40$ 的项集级结果虽有 emitted-circuit ANF 符号等价验证, 但不包含完整 truth-table 枚举。

9 结论

本文提出了面向资源约束量子布尔函数综合的 Resource-NMCTS 方法。它以 ANF/FPRM 项集合为状态, 以逻辑资源 score 为目标, 用布尔环线性因子、神经动作先验、MCTS、baseline guard、depth policy、depth-frontier policy 和 screen-gate 共同生成候选线路。实验表明, 该方法在 $n \leq 6$ 传统函数上显著降低 T-count 与加权 score, 在 $n = 18, 20$ 高维函数级切片上验证了 adaptive screen 与 Resource tail 的互补作用, 在 $n = 20$ 到 $n = 40$ 项集级 scale 测试中展示了结构级 AI 的可扩展性, 并通过 depth-frontier 与 depth-frontier policy 证明更深 Boolean-ring screen 可在高预算下继续降低 score 且能被学习策略部分吸收。 $n = 21, 22$ truth-table bridge 进一步证明生成线路在完整函数枚举下仍保持正确。当前稿件已经形成独立于 SSHR 的论文主线, 但最终投稿还需要继续强化高维神经策略的独立质量收益和外部工具链比较。

参考文献

- [1] G. Meuli, M. Soeken, E. Campbell, M. Roetteler, and G. De Micheli. The role of multiplicative complexity in compiling low T-count oracle circuits. In *Proceedings of ICCAD*, 2019.
- [2] G. Meuli, M. Soeken, and G. De Micheli. Xor-And-Inverter Graphs for quantum compilation. *npj Quantum Information*, 8:7, 2022.
- [3] G. Meuli, M. Soeken, M. Roetteler, and G. De Micheli. ROS: Resource-constrained oracle synthesis for quantum computers. *Electronic Proceedings in Theoretical Computer Science*, 318:119–130, 2020.
- [4] R. Wille and R. Drechsler. BDD-based synthesis of reversible logic for large functions. In *Proceedings of DAC*, pp. 270–275, 2009.
- [5] M. Yu, A. Tempia Calvino, M. Soeken, and G. De Micheli. Back-end-aware fault-tolerant quantum oracle synthesis. In *Proceedings of ASP-DAC*, pp. 930–937, 2025.
- [6] Q. Zheng, Y. Xu, J. Zhang, Z. Su, and S. Zheng. CNOT oriented synthesis for small-scale Boolean functions using spatial structures of parallelotopes. In *Proceedings of ICCAD*, 2025.
- [7] D. Tsaras, A. Grosnit, L. Chen, Z. Xie, H. Bou-Ammar, and M. Yuan. ShortCircuit: AlphaZero-driven circuit design. *arXiv:2408.09858*, 2024.
- [8] S. Rietsch, A. Y. Dubey, C. Ufrecht, M. Periyasamy, A. Plinge, C. Mutschler, and D. D. Scherer. Unitary synthesis of Clifford+T circuits with reinforcement learning. In *IEEE International Conference on Quantum Computing and Engineering*, pp. 824–835, 2024.

- [9] J. Riu, J. Nogu  , G. Vilaplana, A. Garcia-Saez, and M. P. Estarellas. Reinforcement learning based quantum circuit optimization via ZX-calculus. *Quantum*, 9:1758, 2025.
- [10] M. Machiya, M. Menickelly, P. Hovland, and J. Liu. MonteQ: A Monte Carlo tree search based quantum circuit synthesis framework. *arXiv:2604.19029*, 2026.